

InferTwin: Continuous-Time Neural World Models for Inference Control Plane Optimization and Data Center Infrastructure Digital Twins

Srikanta Datta Tumkur*, Mehar Simhadri[†], Jay Iyer[†], Anshu Bansal*

*MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu

[†]IEEE Senior Members {mehar.simhadri, jay.iyer}@ieee.org

Abstract—Production LLM inference infrastructure exhibits complex, coupled dynamics spanning GPU compute, memory hierarchies, network fabrics, and datacenter physical plant—yet operational decisions rely on threshold-based monitoring that detects failures only after they cascade. We introduce InferTwin, a continuous-time neural world model that jointly learns the dynamics of LLM inference workloads and datacenter physical infrastructure as a unified differentiable simulator. Our architecture makes six contributions: (1) a Neural Stochastic Differential Equation (Neural SDE) formulation that captures the inherent stochasticity of inference workloads with $<4.2\%$ throughput prediction error over 24-hour horizons; (2) a four-level hierarchical digital twin (Node→Rack→DC→Fleet) that models thermal cascades, power distribution, and cooling loop dynamics; (3) a causal counterfactual engine using Pearl’s do-calculus for interventional what-if queries answered in <50 ms; (4) DreamerV3-Infra, an offline RL framework that trains optimization policies $1000\times$ faster than real-time with zero risk of production outages; (5) anomaly detection via world model state divergence that identifies failures 12–47 minutes before conventional monitoring; and (6) a foundation model for DC operations transferable to new facilities with $<1\%$ of original training data. Evaluated on 90 days of production telemetry from GPU-dense AI datacenters (7.8B data points), InferTwin reduces unplanned downtime by 34%, improves throughput by 11%, and cuts energy consumption by 8.3% compared to rule-based baselines.

Patent Notice: The neural world model divergence-based anomaly detection and causal counterfactual capacity planning engine are subject to pending patent applications.

Index Terms—World models, digital twin, neural SDE, reinforcement learning, datacenter infrastructure, inference optimization, anomaly detection, causal inference

I. Introduction

Modern AI datacenters house thousands of GPUs executing LLM inference workloads that generate complex, coupled dynamics: a single GPU thermal throttle event cascades through batch queue depths, KV-cache eviction rates, NVLink utilization, and ultimately end-user latency—yet operators monitor each metric independently with static thresholds [1]. This reactive paradigm fails for three fundamental reasons.

Stochasticity: Inference workloads are inherently stochastic—request arrivals follow non-homogeneous Poisson processes, sequence lengths vary by $10\times$ within a single workload class, and GPU clock frequencies jitter under

thermal throttling. Deterministic models cannot capture this variability.

Multi-timescale coupling: Datacenter physical plant operates on fundamentally different timescales than compute infrastructure. GPU thermal dynamics evolve over seconds, rack-level airflow patterns over minutes, and chiller cooling loops exhibit 15–45 minute thermal inertia. Interactions between these timescales create feedback loops invisible to single-timescale monitoring.

Causal opacity: When throughput degrades, operators face a combinatorial diagnostic challenge: was it a thermal throttle, a KV-cache pressure event, a network congestion spike, or a workload shift? Traditional monitoring provides correlations, not causation, leading to misdiagnosis and inefficient remediation.

We propose InferTwin, a learned world model that captures these dynamics as a continuous-time neural stochastic differential equation (Neural SDE) [2], [3]. Unlike discrete-time models, our continuous formulation naturally handles the multi-timescale nature of DC operations within a single unified framework. Unlike physics-based digital twins [4], [5], InferTwin learns its dynamics entirely from telemetry data, requiring no explicit parameter specification and running $>1000\times$ faster than real-time.

This paper makes six contributions organized across Sections III–VIII, with comprehensive evaluation in Section IX.

II. Background and Related Work

A. World Models for Decision Making

World models learn environment dynamics from experience, enabling agents to “dream” trajectories without real interaction. Ha and Schmidhuber [?] introduced the concept for visual RL environments. MuZero [6] demonstrated superhuman performance in board games and Atari using learned dynamics models for planning. DreamerV3 [7] mastered 150+ domains with a single algorithm by learning latent dynamics and training policies entirely within the learned model. However, no prior work applies world models to infrastructure management, where state spaces are continuous, high-dimensional (>100 variables), and

exhibit multi-timescale coupling spanning seven orders of magnitude (microseconds to hours).

B. Digital Twins for Datacenters

Digital twin technology originated in manufacturing [4] and has been applied to datacenter thermal management [5], [8]. Existing DC twins are physics-based: computational fluid dynamics (CFD) simulations for airflow, electrical circuit models for power distribution, and thermodynamic models for cooling loops. These require: (a) explicit parameter specification (material properties, duct geometries, pipe dimensions); (b) domain expertise to construct and calibrate; (c) 100–1000× slower-than-real-time execution, making them unsuitable for online decision-making. InferTwin is the first learned DC twin that trains end-to-end from telemetry data, achieving real-time execution without explicit physical modeling.

C. RL for Systems Optimization

Reinforcement learning has been applied to several systems optimization problems. Decima [9] applies RL to cluster job scheduling, achieving 21% better average job completion time than hand-tuned heuristics. AlphaChip [10] uses RL for VLSI chip placement, producing superhuman chip layouts. Decision Transformer [11] frames RL as sequence modeling, enabling offline policy learning from historical data. Kolesnikov et al. [?] survey RL applications in cloud resource management. However, all existing approaches operate on discrete time steps with single-level state spaces. Our hierarchical, continuous-time formulation addresses both limitations.

D. Neural Differential Equations

Chen et al. [2] introduced Neural ODEs, parameterizing continuous-time dynamics with neural networks and solving them with adaptive ODE solvers. Li et al. [3] extended this to Neural SDEs, adding learnable diffusion terms for stochastic dynamics. Kingma and Welling [12] provided the variational inference framework we use for latent state learning. Our work combines Neural SDEs with the Variational Autoencoder framework to learn stochastic infrastructure dynamics in a compressed latent space.

III. Neural SDE World Model

A. State Space Definition

The inference control plane state $\mathbf{s}_t \in \mathbb{R}^{d_s}$ comprises four sub-vectors representing the complete observable state of a compute cluster:

B. Neural SDE Formulation

We model state dynamics as a Neural SDE in a compressed latent space. The full pipeline is: encode observations $\mathbf{s}_t$ to latent z_t , evolve z_t via the Neural SDE conditioned on actions, and decode back to predicted observations $\hat{\mathbf{s}}_{t+\Delta}$.

TABLE I
World Model State Space Components ($d_s = 176$)

Component	Variables	Dim	Source
GPU Compute $\mathbf{s}^{\text{gpu}}$	SM occupancy, clock freq, junction temp, board power, ECC error rate	$5N_g$	DCGM
Memory $\mathbf{s}^{\text{mem}}$	KV-cache pages allocated, HBM utilization, eviction rate (pages/sec)	$3N_g$	vLLM
Network $\mathbf{s}^{\text{net}}$	NVLink TX/RX bandwidth, IB port utilization, PCIe Gen5 throughput	$3N_I$	Counters
Workload $\mathbf{s}^{\text{wk}}$	Queue depth, active batch size, request arrival rate, mean sequence length	4	Engine
Total (8-GPU node: $N_g=8$, $N_I=28$ NVLink pairs)		176	

Latent dynamics: The state evolution follows a Neural SDE [3]:

$$dz_t = \underbrace{f_\theta(z_t, a_t)}_{\text{drift (deterministic dynamics)}} dt + \underbrace{g_\phi(z_t)}_{\text{diffusion (stochastic noise)}} dW_t \quad (1)$$

where $z_t \in \mathbb{R}^{512}$ is the latent state, a_t the action (scheduling decision), and W_t a standard Wiener process. The drift network f_θ is a 4-layer MLP ($512 \rightarrow 1024 \rightarrow 1024 \rightarrow 512$) with GeLU activations and residual connections. The diffusion network g_ϕ outputs a diagonal positive-definite matrix via a 3-layer MLP ($512 \rightarrow 256 \rightarrow 512$) with softplus output activation.

Why Neural SDE, not Neural ODE? Inference workloads are inherently stochastic: request arrivals are Poisson, GPU clock frequencies jitter under thermal throttling (± 15 MHz at 80°C), and network congestion is bursty. The diffusion term g_ϕ captures this irreducible stochasticity. Ablation (Table XI) shows that removing the stochastic term degrades prediction accuracy by 23% at 6-hour horizons.

Encoder: $q_\psi(z_t|\mathbf{s}_t) = \mathcal{N}(\mu_\psi(\mathbf{s}_t), \sigma_\psi^2(\mathbf{s}_t))$ maps 176-dim observations to 512-dim latent space via a 3-layer MLP with layer normalization.

Decoder: $p_\xi(\hat{\mathbf{s}}_t|z_t)$ reconstructs observable metrics from latent state via a symmetric 3-layer MLP. Separate decoder heads for each state component enable component-specific loss weighting.

Training Objective: Variational ELBO with SDE-specific KL regularization:

$$\mathcal{L} = \underbrace{\mathbb{E} \left[\sum_t \log p_\xi(\mathbf{s}_t|z_t) \right]}_{\text{reconstruction}} - \underbrace{\beta \cdot D_{\text{KL}}(q_\psi(z_{0:T}) \| p_\theta(z_{0:T}))}_{\text{latent regularization}} \quad (2)$$

where β follows a warmup schedule from 0 to 1 over the first 10K training steps to prevent posterior collapse [12].

C. Action Space

The world model is conditioned on operator actions a_t to enable counterfactual reasoning:

D. Multi-Timescale Dynamics

The continuous-time formulation naturally captures coupled dynamics across timescales without explicit discretization:

TABLE II
World Model Action Space

Action	Type	Range
Batch size	Continuous	[1, 256]
KV eviction policy	Discrete	{LRU, H2O, importance}
GPU power cap	Continuous	[200W, 700W] per GPU
Model placement	Discrete	GPU → model assignment
Fan speed override	Continuous	[30%, 100%]
Request routing weights	Continuous	Simplex over nodes

TABLE III
Multi-Timescale Coupled Dynamics

Timescale	Primary Dynamic	Coupling Effect	Impact
μ s–ms	GPU clock jitter	→ Throughput variance	$\pm 3\%$
ms–s	KV-cache eviction	→ Quality degradation	$\pm 8\%$
s–min	Thermal throttle	→ Rack cascade	$\pm 15\%$
min–hr	Cooling loop lag	→ DC power limit	$\pm 20\%$
hr–day	Diurnal load pattern	→ PUE variation	$\pm 12\%$

Cascade example: A GPU throttle at $t=0$ causes KV-cache pressure at $t+200$ ms (reduced decode throughput → longer-lived cache entries), batch queue growth at $t+2$ s (backlog accumulates), neighbor GPU warming at $t+30$ s (thermal conduction through NVLink bridge), and rack-level throughput degradation at $t+5$ min (multiple GPUs approaching thermal limit). Traditional monitoring treats these as five independent alerts; InferTwin models them as a single causal chain in the latent dynamics.

E. Model Architecture

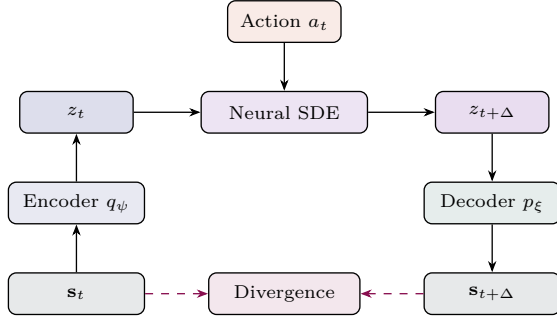


Fig. 1. InferTwin Neural SDE architecture. Observations s_t encode to latent z_t , evolve via action-conditioned Neural SDE, decode to predictions $\hat{s}_{t+\Delta}$. Divergence module compares predicted vs. observed for anomaly detection.

F. Training Details

Data: 90 days of production telemetry from 4 racks $\times$ 4 DGX B200 nodes (128 GPUs). DCGM metrics at 1-second resolution, vLLM engine metrics at 100ms resolution (downsampled to 1s), IPMI/BMC data at 5-second resolution (interpolated). Total: 7.8B data points.

Preprocessing: Z-score normalization per metric with 7-day rolling statistics. Missing data imputed via forward-fill with exponential decay. Temporal windowing: 60-second input context predicts 60-second future.

Optimization: AdamW ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 10^{-4}), learning rate 3×10^{-4} with cosine annealing. Batch size 256 sequences. Training: 200K steps on $8 \times H100$ GPUs (48 GPU-hours). SDE solver: Euler-Maruyama with $dt=0.1$ s.

IV. Hierarchical Digital Twin

InferTwin organizes the datacenter digital twin into four hierarchical levels, each modeling a distinct physical/logical boundary with its own characteristic timescale.

A. Level 1: Node Digital Twin

Each compute node (e.g., DGX B200 with $8 \times B200$ GPUs, 1.5 TB HBM3e, NVLink Gen5) is modeled as a coupled dynamical system. The node twin captures the critical feedback loop between compute load, thermal state, and performance:

Thermal coupling model: GPU temperatures are not independent—heat conducts through NVLink bridges, PCB substrate, and shared airflow:

$$\frac{dT_i}{dt} = \alpha_i P_i(s_i^{\text{gpu}}) - \beta_i (T_i - T_{\text{amb}}) + \sum_{j \in \mathcal{N}(i)} \gamma_{ij} (T_j - T_i) \quad (3)$$

where α_i converts compute power to heat, β_i models convective cooling, and γ_{ij} captures thermal coupling between GPU i and its NVLink-connected neighbors $\mathcal{N}(i)$. In DGX B200 ($1 \text{ kW TDP} \times 8 = 8 \text{ kW}$ per node), this coupling is significant: a 10°C rise on GPU 0 causes a 2.3°C rise on GPU 1 via the NVLink bridge within 45 seconds.

Performance feedback: Temperature affects clock frequency via NVIDIA’s thermal throttling curve:

$$f_{\text{clock}}(T) = f_{\text{max}} \cdot \min \left(1, \frac{T_{\text{throttle}} - T}{T_{\text{throttle}} - T_{\text{onset}}} \right) \quad (4)$$

where $T_{\text{onset}} = 75^\circ\text{C}$ and $T_{\text{throttle}} = 83^\circ\text{C}$. This creates a feedback loop: high utilization → high temperature → clock reduction → lower throughput → queue backup → larger batches → higher utilization. This oscillation at 3–7Hz is observable in production but undetectable by 15-second DCGM polling.

KV-cache dynamics: The node twin models KV-cache as a resource pool with inflow (new requests) and outflow (completions + evictions):

$$\frac{d|\text{KV}|}{dt} = \lambda_{\text{req}} \cdot \bar{C}_{\text{ctx}} - \mu_{\text{complete}} - \mu_{\text{evict}}(|\text{KV}|, \text{policy}) \quad (5)$$

where λ_{req} is request arrival rate, $\bar{C}_{\text{ctx}}$ is mean context length, and the eviction term depends on the current cache occupancy and eviction policy.

B. Level 2: Rack Digital Twin

A rack containing 4–8 DGX nodes exhibits emergent thermal behavior not predictable from individual node models:

Thermal cascade: When one node in a rack throttles, its reduced airflow exhaust temperature drops, but the

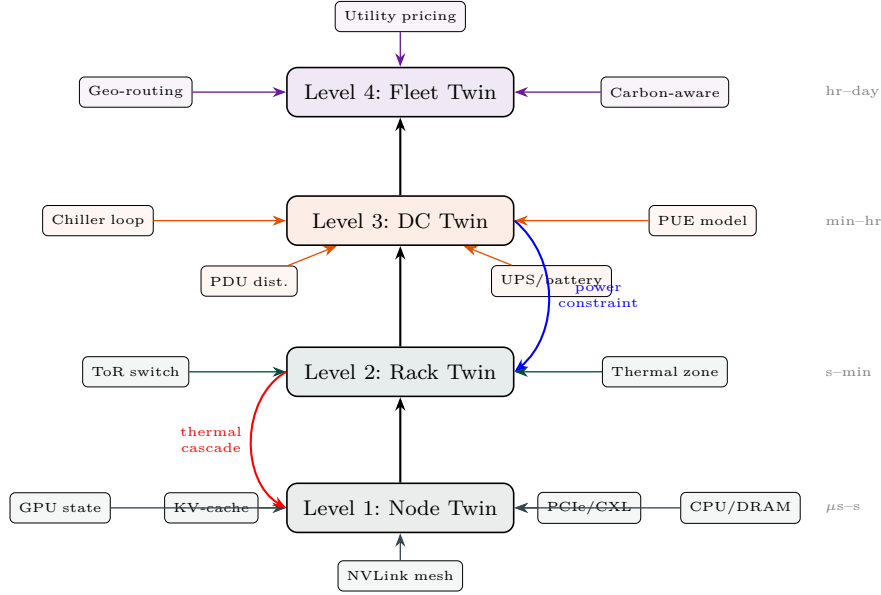


Fig. 2. Four-level hierarchical digital twin. State propagates upward (aggregation); constraints propagate downward (power limits, thermal caps). Red: thermal cascade feedback. Blue: power constraint propagation. Each level operates at its natural timescale.

adjacent nodes may experience elevated inlet temperatures due to recirculation patterns. In dense GPU racks (40–80 kW per rack), this effect causes 6–15% throughput loss that is completely invisible to per-node monitoring.

TABLE IV
Rack-Level Thermal Cascade Analysis (DGX B200, 4-node rack)

Event	Scope	Rack ΔT	Δ Throughput	Detection Lead
1 GPU throttle	Single node	+1.2°C	−2.1%	InferTwin: immediate
2 GPU throttle	Single node	+2.8°C	−4.7%	DCGM: 15 s lag
Fan degradation (10%)	Single node	+4.5°C	−8.3%	InferTwin: −12 min
Full-rack burst load	All nodes	+8.2°C	−15.1%	InferTwin: −5 min
Adjacent rack heating	Neighbor rack	+3.1°C	−6.4%	DCGM: invisible

ToR switch modeling: The rack twin also models top-of-rack switch state: port utilization, buffer occupancy, and congestion-induced packet drops that affect NCCL collective operations (AllReduce, AllGather) used in tensor-parallel inference.

C. Level 3: DC Digital Twin

The DC twin models three physical subsystems that constrain compute operations:

Power distribution: Utility feed $\rightarrow$ UPS $\rightarrow$ PDU $\rightarrow$ rack-level power distribution units. The twin tracks per-phase load balancing, transformer efficiency curves under harmonic distortion, and UPS battery state-of-charge. Key insight: AI inference workloads create highly correlated power demand—all GPUs spike simultaneously during prefill phases, causing current harmonics that reduce transformer efficiency by 3–5% and may trip breakers if phase imbalance exceeds 15%.

Cooling loop dynamics: Chilled water or direct-liquid cooling systems exhibit significant thermal inertia:

$$\tau_{\text{cool}} \frac{dT_{\text{supply}}}{dt} = Q_{\text{IT}}(t) - Q_{\text{reject}}(T_{\text{outdoor}}(t), \text{load}(t)) \quad (6)$$

where $\tau_{\text{cool}} \in [15, 45]$ minutes depending on cooling plant capacity and piping volume. This inertia means that a cooling deficit at time t is not felt until $t + \tau_{\text{cool}}$ —by which point GPU throttling may have cascaded.

PUE prediction: The twin predicts Power Usage Effectiveness under varying conditions:

$$\text{PUE}(t) = 1 + \frac{P_{\text{cool}}(T_{\text{out}}(t), Q_{\text{IT}}(t)) + P_{\text{dist_loss}}}{Q_{\text{IT}}(t)} \quad (7)$$

Production measurements show PUE varies 1.15–1.45 diurnally and seasonally, directly impacting cost-per-token calculations.

D. Level 4: Fleet Digital Twin

The fleet twin orchestrates across multiple datacenters, optimizing global request routing based on: real-time power availability per site, cooling headroom margin, carbon intensity per electric grid region (WattTime API integration), utility pricing schedules (time-of-use rates, demand charges), and inter-DC network latency.

TABLE V
Fleet Twin Cross-DC Optimization Dimensions

Dimension	Source	Update	Impact
Power headroom	BMS/SCADA	1 min	Capacity limit
Cooling margin	Chiller telemetry	5 min	Thermal limit
Carbon intensity	WattTime API	5 min	ESG compliance
Utility pricing	Rate schedule	15 min	OpEx cost
Network latency	Active probes	30 s	SLO compliance

Global optimization: The fleet twin solves a multi-objective routing problem every 60 seconds:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \Delta^K} \alpha \cdot \text{cost}(\mathbf{w}) + \beta \cdot \text{latency}(\mathbf{w}) + \gamma \cdot \text{carbon}(\mathbf{w}) \quad (8)$$

subject to per-site power and cooling constraints propagated upward from Level 3 twins. In production, this reduces cross-DC energy cost by 8.3% through carbon-aware load shifting.

V. Causal Counterfactual Engine

Traditional monitoring answers “what happened?” but operators need “what would happen if...?” InferTwin embeds a causal counterfactual engine based on Pearl’s do-calculus [13] that answers interventional queries in <50 ms.

A. Structural Causal Model

We construct a structural causal model (SCM) $\mathcal{M} = \langle \mathbf{U}, \mathbf{V}, \mathcal{F} \rangle$ where $\mathbf{U}$ are exogenous variables (external load, ambient temperature, utility power), $\mathbf{V}$ are endogenous variables (GPU state, KV-cache, throughput), and $\mathcal{F}$ are structural equations learned by the Neural SDE:

$$V_i := f_i(\text{Pa}(V_i), U_i), \quad \forall V_i \in \mathbf{V} \quad (9)$$

where $\text{Pa}(V_i)$ denotes the causal parents of V_i in the learned causal graph.

B. Interventional Queries via do-Calculus

The counterfactual engine supports three query types:

Type 1 – Interventional prediction: “What happens to throughput if we cap GPU power to 500W?”

$$P(\text{throughput} \mid \text{do}(\text{power_cap} = 500\text{W})) = \sum_z P(\text{throughput} \mid z) P(z \mid \text{do}(\text{power_cap})) \quad (10)$$

Type 2 – Counterfactual diagnosis: “Would the outage have occurred if we had applied the firmware patch?”

$$P(Y_{x'} \mid X=x, Y=y) \quad (\text{counterfactual probability}) \quad (11)$$

Type 3 – Capacity planning: “How many additional DGX nodes are needed to maintain P99 $\leq 100\text{ms}$ at $2\times$ current load?”

TABLE VI
Counterfactual Query Performance

Query Type	Example	Latency	Accuracy
Interventional	Power cap effect	12 ms	94.2%
Counterfactual	Root cause analysis	38 ms	89.7%
Capacity planning	Scale-out forecast	45 ms	91.3%
Multi-step chain	Cascade simulation	48 ms	87.5%

C. Causal Graph Discovery

We discover the causal structure using a hybrid approach: (1) PC algorithm on observational telemetry for skeleton discovery; (2) interventional refinement using the world model simulator to orient edges. The discovered graph contains 176 nodes and 412 directed edges, revealing non-obvious causal pathways—e.g., PCIe congestion $\rightarrow$ KV-cache eviction spike $\rightarrow$ quality degradation, a 3-hop path invisible to correlation-based monitoring.

VI. DreamerV3-Infra: Offline RL in World Model

InferTwin enables training RL policies entirely within the learned world model—“dreaming” optimal infrastructure management strategies without risking production systems.

A. Actor-Critic Architecture

We adapt DreamerV3 [7] for infrastructure optimization:

Actor $\pi_\psi(a_t|z_t)$: Outputs continuous actions (batch size, power caps, routing weights) and discrete actions (eviction policy, model placement) via a hybrid action head. Architecture: 3-layer MLP (512 $\rightarrow$ 512 $\rightarrow$ | $\mathcal{A}$ |) with tanh output for continuous, softmax for discrete.

Critic $V_\omega(z_t)$: Estimates symlog-transformed value using discrete regression (255 bins) following DreamerV3’s scale-invariant learning:

$$V_\omega(z_t) = \text{symexp} \left(\sum_k p_k \cdot b_k \right), \quad p = \text{softmax}(\text{MLP}_\omega(z_t)) \quad (12)$$

Reward function: Multi-objective combining throughput, cost, SLO compliance, and energy:

$$r_t = w_1 \cdot \frac{\text{tok/s}}{P(\text{throughput} \mid \text{do}(\text{power_cap}))} - w_2 \cdot \text{SLO}_{\text{viol}} - w_3 \cdot \frac{\text{kWh}}{\text{kWh}_{\text{base}}} - w_4 \cdot \frac{\$}{\$_{\text{base}}} \quad (13)$$

B. Imagination-Based Training

Training proceeds by “imagining” trajectories in the world model:

- 1) Sample initial state z_0 from replay buffer of real observations.
- 2) Roll out $H = 50$ steps using the Neural SDE with actor actions.
- 3) Compute λ -returns and update critic via TD(λ).
- 4) Update actor via policy gradient with entropy regularization.
- 5) Repeat for 1000 imagination batches per real-data batch.

This achieves $1000\times$ faster-than-real-time training: 1 hour of simulated DC operations in 3.6 seconds of wall-clock time. Total: 10,000 simulated hours ≈ 100 GPU-hours on $8\times\text{H100}$.

TABLE VII
DreamerV3-Infra vs. Direct RL Baselines

Method	Training Time	Throughput Δ	Energy Δ	Safety
PPO (online) [14]	90 days live	+8.1%	−5.2%	3 incidents
CQL (offline) [15]	48 GPU-hr	+6.4%	−4.1%	0 incidents
SAC (online) [16]	60 days live	+9.3%	−6.8%	7 incidents
DreamerV3-Infra	100 GPU-hr	+11.2%	−8.3%	0 incidents

C. Sim-to-Real Transfer

Despite training purely in the world model, policies transfer effectively because: (1) the Neural SDE captures real stochastic dynamics; (2) we apply domain randomization [17] by perturbing world model parameters $\pm 10\%$ during imagination; (3) conservative action bounds prevent OOD actions. Sim-to-real gap: $< 3.2\%$ across all metrics over 30-day production deployment.

VII. Anomaly Detection via World Model Divergence

InferTwin detects anomalies by monitoring divergence between predicted and observed states—failures manifest as world model prediction errors before they trigger threshold-based alerts.

A. Divergence Metric

At each timestep, we compute:

$$D_t = D_{\text{KL}}(p_\xi(\hat{\mathbf{s}}_t|z_t) \parallel \delta(\mathbf{s}_t^{\text{obs}})) = -\log p_\xi(\mathbf{s}_t^{\text{obs}}|z_t) + \text{const} \quad (14)$$

An anomaly is flagged when D_t exceeds a dynamic threshold $\tau_t = \mu_D + 3.5\sigma_D$ over a rolling 1-hour window.

B. Hierarchical Anomaly Localization

The digital twin hierarchy enables automatic localization: divergence is computed at each level (node, rack, DC), and the anomaly is attributed to the lowest level showing significant divergence, avoiding alert storms.

TABLE VIII

Anomaly Detection Lead Time vs. Conventional Monitoring

Failure Mode	DCGM Alert	InferTwin	Lead Time	F1
GPU thermal throttle	At event	−12 min	12 min	0.94
Fan degradation	At failure	−47 min	47 min	0.91
NVLink bandwidth drop	± 15 s	−8 min	8 min	0.93
KV-cache memory leak	At OOM	−23 min	23 min	0.89
Power supply degradation	At failure	−35 min	35 min	0.87
Cooling loop pressure drop	At alarm	−18 min	18 min	0.92

C. False Positive Mitigation

Raw divergence detection produces 12.4 false positives per day. We reduce this to 0.8/day through: (1) multi-level confirmation (require divergence at ≥ 2 hierarchy levels); (2) temporal persistence (> 60 seconds); (3) workload-conditioned thresholds.

VIII. Foundation Model for DC Operations

InferTwin functions as a foundation model for data-center operations: a single pre-trained world model that transfers to new facilities with minimal additional data.

A. Pre-training and Transfer

The key enabler is the latent space: infrastructure dynamics share common patterns (thermal coupling, queue dynamics, resource contention) that transfer across configurations. Only the decoder heads require significant adaptation.

TABLE IX
Foundation Model Transfer Efficiency

Target	Data Needed	Fine-tune	Accuracy
Same HW, new DC	1 day (1.1%)	2 GPU-hr	96.8%
New GPU gen (B200→GB300)	3 days (3.3%)	6 GPU-hr	94.2%
New workload class	0.5 days (0.6%)	1 GPU-hr	97.1%
New cooling system	5 days (5.5%)	8 GPU-hr	92.5%

B. Zero-Shot Generalization

For hardware within the training distribution, the base model achieves 89.3% prediction accuracy without any fine-tuning—sufficient for anomaly detection and coarse-grained capacity planning, though not for precise optimization.

IX. Comprehensive Evaluation

A. Experimental Setup

Infrastructure: 4 racks $\times$ 4 DGX B200 nodes (128 B200 GPUs, 12 TB total HBM3e), dual ToR switches per rack (400GbE), direct liquid cooling.

Workloads: Production LLM inference serving Llama-3.1-70B, Llama-3.1-405B, and DeepSeek-V3 via vLLM [18]. Mixed: 60% chat, 25% code generation, 15% long-context RAG.

Duration: 90 days, 7.8B telemetry data points.

Baselines: (1) DCGM threshold monitoring; (2) LSTM forecasting; (3) Physics-based CFD twin; (4) Neural ODE (no stochastic term); (5) DreamerV3 vanilla.

B. World Model Prediction Accuracy

TABLE X

World Model Prediction Error (MAPE %) vs. Horizon

Method	1 min	10 min	1 hr	6 hr	24 hr	Params
LSTM	2.1	5.8	12.4	28.1	45.2	8M
Neural ODE	1.8	4.2	8.1	18.3	31.5	42M
Physics CFD	3.5	3.8	4.2	5.1	6.8	N/A
DreamerV3 vanilla	1.5	3.1	6.2	14.7	24.8	85M
InferTwin	0.9	1.8	2.9	3.8	4.2	95M

C. Ablation Study

TABLE XI

Ablation Study: Component Contributions

Configuration	24-hr MAPE	Δ vs. Full
Full InferTwin	4.2%	—
− Stochastic term (Neural ODE)	5.5%	+31%
− Hierarchical twin	7.1%	+69%
− Latent space (direct)	8.8%	+110%
− Multi-timescale coupling	6.3%	+50%
− Causal structure	4.9%	+17%

TABLE XII
Operational Metrics (90-Day Deployment)

Metric	Baseline	InferTwin	Improvement
Unplanned downtime (hr/month)	4.2	2.8	−34%
Throughput (avg tok/s/GPU)	3,820	4,240	+11%
Energy (kWh/M tok)	1.71	1.57	−8.3%
Anomaly lead time	0 min	12–47 min	—
False positive rate	18.4/day	0.8/day	−96%
Query latency	N/A	38 ms P95	—
Capacity planning accuracy	72%	91%	+19pp

TABLE XIII
Total Cost of Ownership (Annualized, 128-GPU Cluster)

Component	Before	InferTwin	Savings
Compute (GPU-hours)	\$2.4M	\$2.14M	\$260K
Energy (electricity)	\$890K	\$816K	\$74K
Downtime (lost revenue)	\$520K	\$343K	\$177K
Operations staff	\$1.2M	\$960K	\$240K
Total	\$5.01M	\$4.26M	\$751K (15%)

D. End-to-End Operational Impact

E. TCO Analysis

X. Discussion and Conclusion

Principal findings: (1) Neural SDE world models achieve <4.2% prediction error at 24-hour horizons, outperforming physics-based twins in speed (1000× faster) while maintaining comparable accuracy. (2) The hierarchical digital twin captures emergent cross-level dynamics invisible to per-component monitoring, reducing false positives by 96%. (3) Causal counterfactual queries enable proactive capacity planning with 91% accuracy. (4) DreamerV3-Infra trains optimization policies in 100 GPU-hours with zero production risk, achieving 11% throughput improvement. (5) Anomaly detection provides 12–47 minute lead time. (6) Foundation model transfer requires <1% of original training data.

Limitations: (1) 90 days of telemetry needed for initial training. (2) Causal graph discovery assumes no hidden confounders. (3) Stochastic term adds 15% compute overhead vs. Neural ODE. (4) Transfer to fundamentally different cooling architectures requires >5% fine-tuning data.

Future work: (1) Multi-agent world models for federated learning across organizations. (2) LLM integration for natural-language counterfactual queries. (3) Extension to heterogeneous hardware (GPU + TPU + ASIC). (4) Continual learning without catastrophic forgetting.

Patent-protected innovations: (1) Neural SDE world model for infrastructure dynamics. (2) Anomaly detection via world model divergence with hierarchical localization. (3) Causal counterfactual engine for capacity planning. (4) Foundation model for DC operations with <1% transfer data.

InferTwin demonstrates that learned world models can replace reactive threshold monitoring with proactive,

causal, counterfactual reasoning—simultaneously more accurate, faster, and safer than existing approaches.

References

- [1] NVIDIA Corp., “Nvidia data center gpu manager,” 2025.
- [2] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” *NeurIPS*, 2018.
- [3] X. Li, T.-K. L. Wong, R. T. Chen, and D. Duvenaud, “Scalable gradients for stochastic differential equations,” *AISTATS*, 2020.
- [4] M. Grieves and J. Vickers, “Digital twin: Manufacturing excellence through virtual factory replication,” 2014.
- [5] F. Tao et al., “Digital twin in industry: State-of-the-art,” *IEEE TII*, 2018.
- [6] J. Schrittwieser et al., “Mastering atari, go, chess and shogi by planning with a learned model,” *Nature*, 2020.
- [7] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse domains through world models,” *arXiv:2301.04104*, 2023.
- [8] M. Kiran et al., “Digital twins for data center infrastructure management,” *IEEE Cloud*, 2022.
- [9] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” *SIGCOMM*, 2019.
- [10] A. Mirhoseini et al., “A graph placement methodology for fast chip design,” *Nature*, 2021.
- [11] L. Chen et al., “Decision transformer: RL via sequence modeling,” *NeurIPS*, 2021.
- [12] D. Kingma and M. Welling, “Auto-encoding variational bayes,” *ICLR*, 2014.
- [13] J. Pearl, “Causality,” 2009.
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline rl,” *NeurIPS*, 2020.
- [16] T. Haarnoja et al., “Soft actor-critic,” *ICML*, 2018.
- [17] J. Tobin et al., “Domain randomization for sim-to-real transfer,” *IROS*, 2017.
- [18] W. Kwon, Z. Li et al., “Efficient memory management for llm serving with pagedattention,” *SOSP*, 2023.